

The pandas Library

Objectives

- ✓ pandas
- ✓ pandas data structure
- ✓ Indexes
- ✓ Opeartions
- ✓ Sorting and Ranking
- ✓ Correlation and Covariance

pandas

- The **pandas** library is a powerful Python tool for data analysis.
- It introduces how to install pandas and explains its two main data structures: **Series** and **DataFrame**.
- Introduces key concepts such as **indexing** for data manipulation and extends this to **hierarchical indexing**, which allows multiple levels of indexing for more advanced data organization.

- pandas is an open-source Python library widely used for data analysis, statistics, and decision-making.
- Provide simple yet powerful tools for data processing, extraction, and manipulation.
- Built on top of NumPy, pandas benefits from strong compatibility and performance.
- It introduces two specialized data structures designed for relational and labeled data, similar to SQL databases and Excel spreadsheets.
- Overall, pandas provides essential tools and functions for efficient data analysis.

pandas data structure

- pandas is built around two core data structures: **Series** and **DataFrame**.
- A Series handles one-dimensional data, while a DataFrame manages multi-dimensional data.
- Though not a universal solution, these structures are simple, robust, and suitable for most data analysis tasks. Their key strength lies in the integration of **index objects and labels**, which makes data manipulation efficient and intuitive.

The series

- a one-dimensional data structure similar to an array but with added features.
- It consists of two linked arrays: one that stores the data (of any NumPy type) and another that stores corresponding labels called the **index**.
- These labels make it easier to organize and access the data.

Series	
index	value
0	12
1	-4
2	7
3	9

- Declaring a Series

```
import numpy as np
import pandas as pd
#from pandas import *
```

```
s = pd.Series([12,-4,7,9])
```

s

	0
0	12
1	-4
2	7
3	9

dtype: int64

```
#create a series using meaningful labels
s = pd.Series([12,-4,7,9], index=['a','b','c','d'])
```

s

	0
a	12
b	-4
c	7
d	9

dtype: int64

- Selecting the Internal Elements

```
#select individual elements as ordinary NumPy arrays, specifying the key.  
s['c'] #if no label: s[2]
```

```
np.int64(7)
```

```
#select multiple items in a NumPy array  
s[0:2]
```

```
0
```

```
a 12
```

```
b -4
```

```
dtype: int64
```

```
s[['b', 'c']]
```

```
0
```

```
b -4
```

```
c 7
```

```
dtype: int64
```


- Assigning Values to the Elements

```
s['b'] = 0
s
```

	0
a	12
b	0
c	7
d	9

dtype: int64

- Defining a Series from NumPy Arrays and Other Series

```
arr[2] = -2 [1,2,3,4])
```

```
s2 arr)
```

	0
0	1
1	2
2	-2
3	4

dtype: int64

dtype: int64

```
s3 = pd.Series(s)
```

```
s3
```

	0
a	12
b	0
c	7
d	9

dtype: int64

```
arr[2] = -2
```

```
s2
```

	0
0	1
1	2
2	-2
3	4

dtype: int64

- Filtering Values

s	
0	
a	12
b	-4
c	7
d	9
dtype: int64	

s[s > 8]	
0	
a	12
d	9
dtype: int64	

- Operations and Mathematical Functions

s	
0	
a	12
b	4
c	7
d	9
dtype: int64	

```
#operators: +, -, *, and /  
s / 2
```

0	
a	6.0
b	2.0
c	3.5
d	4.5
dtype: float64	

```
np.log(s)
```

0	
a	2.484907
b	1.386294
c	1.945910
d	2.197225
dtype: float64	

• Evaluating Values

- There are often duplicate values in a series. Then you may need to have more information about the samples, including existence of any duplicates and whether a certain value is present in the series.

```
serd = pd.Series([1,0,2,1,2,3], index=['white','white','blue','green','green','yellow'])
```

serd

	0
white	1
white	0
blue	2
green	1
green	2
yellow	3

dtype: int64

```
#the unique() function: excluding duplicates  
serd.unique()
```

```
array([1, 0, 2, 3])
```

```
# value_counts(): only returns unique values but also  
#calculates the occurrences within a series.  
serd.value_counts()
```

count

1	2
2	2
0	1
3	1

dtype: int64

- `isin()` function evaluates the membership, that is, the given list of values.
 - This function tells you if the values are contained in the data structure. Boolean values that are returned can be very useful when filtering data in a series or in a column of a dataframe.

```
serd
    0
white 1
white 0
blue  2
green 1
green 2
yellow 3
dtype: int64
```

```
serd.isin([0,3])
    0
white False
white  True
blue  False
green False
green False
yellow  True
dtype: bool
```

```
serd[serd.isin([0,3])]
    0
white 0
yellow 3
dtype: int64
```

- nan Values: **nan (Not a Number)** represents missing or undefined numerical values.
 - It can appear when performing invalid operations (like taking the logarithm of a negative number), when data sources are incomplete, or when errors occur during calculations.
 - Although nan values can cause problems in data analysis and must be handled carefully, pandas allows users to explicitly define and insert them (using `np.nan`) to represent missing data within structures like a Series

```
s2 = pd.Series([5,-3,np.nan,14])
```

s2

	0
0	5.0
1	-3.0
2	NaN
3	14.0

dtype: float64

- The `isnull()` and `notnull()` functions are very useful for identifying the indexes without a value.

```
s2.isnull()
```

	0
0	False
1	False
2	True
3	False

dtype: bool

```
s2.notnull()
```

	0
0	True
1	True
2	False
3	True

dtype: bool

```
s2[s2.isnull()]
```

	0
2	NaN

dtype: float64

```
s2[s2.notnull()]
```

	0
0	5.0
1	-3.0
3	14.0

dtype: float64

These functions are often placed inside filters to make a condition.

- Series as Dictionaries

- create a series from a previously defined dict.

```
mydict = {'red': 2000, 'blue': 1000, 'yellow': 500, 'orange': 1000}
```

```
myseries = pd.Series(mydict)
```

myseries

	0
red	2000
blue	1000
yellow	500
orange	1000

dtype: int64

When creating a pandas structure from a dictionary, the dictionary keys become the **index labels**, and the values become the data. You can also define the index separately. If the provided index labels do not match the dictionary keys, pandas will assign **NaN** to any missing or unmatched entries.

```
colors = ['red', 'yellow', 'orange', 'blue', 'green']
```

```
myseries = pd.Series(mydict, index=colors)
```

myseries

	0
red	2000.0
yellow	500.0
orange	1000.0
blue	1000.0
green	NaN

dtype: float64

- Operations Between Series

- Besides performing arithmetic operations between a Series and a scalar, pandas also allows operations between two Series. In this case, **index labels are used to align the data automatically**. When two Series share some common labels, operations are performed only on matching labels, while non-matching labels result in **NaN** values. This automatic alignment is a key strength of pandas data structures.

myseries		myseries2		myseries + myseries2	
	0		0		0
red	2000.0	red	400	black	NaN
yellow	500.0	yellow	1000	blue	NaN
orange	1000.0	black	700	green	NaN
blue	1000.0			orange	NaN
green	NaN			red	2400.0
				yellow	1500.0
dtype: float64		dtype: int64		dtype: float64	

The Dataframe

- A tabular data structure very similar to a spreadsheet.
- This data structure is designed to extend series to multiple dimensions. In fact, the dataframe consists of an ordered collection of columns, each of which can contain a value of a different type (numeric, string, Boolean, etc.).

DataFrame			
index	columns		
	color	object	price
0	blue	ball	1.2
1	green	pen	1.0
2	yellow	pencil	0.6
3	red	paper	0.9
4	white	mug	1.7

- Defining a Dataframe

```
data = {'color' : ['blue','green','yellow','red','white'],  
        'object' : ['ball','pen','pencil','paper','mug'],  
        'price' : [1.2,1.0,0.6,0.9,1.7]}
```

```
frame = pd.DataFrame(data)
```

frame

	color	object	price	
0	blue	ball	1.2	
1	green	pen	1.0	
2	yellow	pencil	0.6	
3	red	paper	0.9	
4	white	mug	1.7	

- If the dict object from which you want to create a dataframe contains more data than you are interested in, you can make a selection. In the constructor of the dataframe, you can specify a sequence of columns using the columns option.

```
frame2 = pd.DataFrame(data, columns=['object', 'price'])
```

frame2

	object	price
0	ball	1.2
1	pen	1.0
2	pencil	0.6
3	paper	0.9
4	mug	1.7

- To assign labels to the indexes of a dataframe: use the index option and assign it an array containing the labels.

```
frame2 = pd.DataFrame(data, index=['one', 'two', 'three', 'four', 'five'])
```

frame2

	color	object	price
one	blue	ball	1.2
two	green	pen	1.0
three	yellow	pencil	0.6
four	red	paper	0.9
five	white	mug	1.7

- A DataFrame can be created not only from a dictionary but also by passing three arguments to the constructor: a **data matrix**, an **index label array**, and a **column name array**. This provides an alternative and flexible way to define a DataFrame. To quickly generate a data matrix, NumPy's `np.arange(16).reshape((4,4))` can be used to create a 4×4 matrix with values from 0 to 15.

```
frame3 = pd.DataFrame(np.arange(16).reshape((4,4)),  
                      index=['red','blue','yellow','white'],  
                      columns=['ball','pen','pencil','paper'])
```

frame3

	ball	pen	pencil	paper
red	0	1	2	3
blue	4	5	6	7
yellow	8	9	10	11
white	12	13	14	15

• Selecting Elements

frame

	color	object	price
0	blue	ball	1.2
1	green	pen	1.0
2	yellow	pencil	0.6
3	red	paper	0.9
4	white	mug	1.7



```
#the name of all the columns of a dataframe  
frame.columns
```

```
Index(['color', 'object', 'price'], dtype='object')
```

```
# get the list of indexes,  
frame.index
```

```
RangeIndex(start=0, stop=5, step=1)
```

```
#get the entire set of data contained within  
#the data structure using the values attribute  
frame.values
```

```
array([[ 'blue', 'ball', 1.2],  
       [ 'green', 'pen', 1.0],  
       [ 'yellow', 'pencil', 0.6],  
       [ 'red', 'paper', 0.9],  
       [ 'white', 'mug', 1.7]], dtype=object)
```

```
#selecting only the contents of a column: write  
#the name of the column.  
frame['price']
```

price

0	1.2
1	1.0
2	0.6
3	0.9
4	1.7

dtype: float64

```
#or use the column name as an attribute of  
#the instance of the dataframe.  
frame.price
```

price

0	1.2
1	1.0
2	0.6
3	0.9
4	1.7

dtype: float64

```
#For rows within a dataframe, it is possible to use  
#the loc attribute with the index value of the row  
frame.loc[2]
```

	2
colors	yellow
object	pencil
price	0.6

dtype: object

```
#The object returned is again a series in which  
#the names of the columns have become the label of the  
#array index, and the values have become the data of series.  
frame.loc[[2,4]]
```

	colors	object	price
2	yellow	pencil	0.6
4	white	mug	1.7

```
#extract a portion of a dataframe  
frame.iloc[0:1, 0:3]
```

	colors	object	price
0	blue	ball	1.2

```
#more than one line: extend the selection range.  
frame.iloc[1:3,]
```

	colors	object	price
1	green	pen	1.0
2	yellow	pencil	0.6

```
#to achieve is a single value within a dataframe,  
#use the name of the column and then the index or  
#the label of the row.  
frame['object'][3]
```

'paper'

• Assigning Values

- After learning how to access elements in a DataFrame, you can use the same logic to modify or add values. A DataFrame includes an **index** (row labels) and **columns** (column names), and both of these substructures can also be given a label using the **name** attribute for better identification and organization.

frame			
	color	object	price
0	blue	ball	1.2
1	green	pen	1.0
2	yellow	pencil	0.6
3	red	paper	0.9
4	white	mug	1.7

```
frame.index.name = 'id'
```

```
frame.columns.name = 'item'
```

frame

	item	color	object	price
id				
0	blue	ball	1.2	
1	green	pen	1.0	
2	yellow	pencil	0.6	
3	red	paper	0.9	
4	white	mug	1.7	

- Assigning a value to the instance of the dataframe and specifying a new column name.

```
frame['new'] = 12
```

frame

item	color	object	price	new
id				
0	blue	ball	1.2	12
1	green	pen	1.0	12
2	yellow	pencil	0.6	12
3	red	paper	0.9	12
4	white	mug	1.7	12

- Update the contents of a column: use an array.

```
frame['new'] = [3.0,1.3,2.2,0.8,1.1]
```

frame

item	color	object	price	new
id				
0	blue	ball	1.2	3.0
1	green	pen	1.0	1.3
2	yellow	pencil	0.6	2.2
3	red	paper	0.9	0.8
4	white	mug	1.7	1.1

- The columns of a dataframe can also be created by assigning a series to one of them
- Example by specifying a series containing an increasing series of values through the use of np.arange().

```
ser = pd.Series(np.arange(5))
```

ser

	0
0	0
1	1
2	2
3	3
4	4

dtype: int64

```
frame['new'] = ser
```

frame

	item	color	object	price	new
	id				
0	blue	ball	1.2	0	
1	green	pen	1.0	1	
2	yellow	pencil	0.6	2	
3	red	paper	0.9	3	
4	white	mug	1.7	4	

- To access the element, I could think of inserting the column and then the row indexes and thus obtaining the current value

frame

item	color	object	price	new
id				
0	blue	ball	1.2	0
1	green	pen	1.0	1
2	yellow	pencil	0.6	2
3	red	paper	0.9	3
4	white	mug	1.7	4

```
frame['price'][2]
```

```
np.float64(0.6)
```

```
frame.loc[ 2, 'price'] = 2.2
```

frame

item	color	object	price	new
id				
0	blue	ball	1.2	0
1	green	pen	1.0	1
2	yellow	pencil	2.2	2
3	red	paper	0.9	3
4	white	mug	1.7	4

• Membership of a Value

- the `isin()` function applied to the series to determine the membership of a set of values.

frame

	item	color	object	price	new
id					
0	blue	ball	1.2	0	
1	green	pen	1.0	1	
2	yellow	pencil	2.2	2	
3	red	paper	0.9	3	
4	white	mug	1.7	4	

```
frame.isin([1.0, 'pen'])
```

	item	color	object	price	new
id					
0	False	False	False	False	False
1	False	True	True	True	True
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False

```
# get a dataframe containing Boolean values  
frame[frame.isin([1.0, 'pen'])]
```

	item	color	object	price	new
id					
0	NaN	NaN	NaN	NaN	NaN
1	NaN	pen	1.0	1.0	1.0
2	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN

- **Deleting a Column**

- Use the del command

frame				
item	color	object	price	new
id				
0	blue	ball	1.2	0
1	green	pen	1.0	1
2	yellow	pencil	2.2	2
3	red	paper	0.9	3
4	white	mug	1.7	4

```
del frame['new']
```

frame			
item	color	object	price
id			
0	blue	ball	1.2
1	green	pen	1.0
2	yellow	pencil	2.2
3	red	paper	0.9
4	white	mug	1.7

• Filtering

- Even with a dataframe, you can apply filtering through the application of certain conditions.

id	item	color	object	price
0	blue	ball	1.2	
1	green	pen	1.0	
2	yellow	pencil	2.2	
3	red	paper	0.9	
4	white	mug	1.7	

```
frame[frame['price'] < 1.2]
```

id	item	color	object	price
1	green	pen	1.0	
3	red	paper	0.9	

- **Dataframe from a Nested dict**

- A very common data structure used in Python is a nested dict

```
nestdict = {'red':{2012: 22, 2013: 33},  
            'white':{2011: 13, 2012: 22, 2013: 16},  
            'blue': {2011: 17, 2012: 27, 2013: 18}}
```

nestdict

```
{'red': {2012: 22, 2013: 33},  
 'white': {2011: 13, 2012: 22, 2013: 16},  
 'blue': {2011: 17, 2012: 27, 2013: 18}}
```

```
frame2 = pd.DataFrame(nestdict)
```

frame2

	red	white	blue
2012	22.0	22	27
2013	33.0	16	18
2011	NaN	13	17



- **Transposition of a Dataframe:**

- Get the transposition of the dataframe by adding the T attribute

frame2.T

	2012	2013	2011
red	22.0	33.0	NaN
white	22.0	16.0	13.0
blue	27.0	18.0	17.0

The Index Objects

- The key features of Series and DataFrame largely come from the integrated **Index object**. The Index manages axis labels and related metadata, such as axis names. When creating these structures, label arrays are automatically converted into Index objects, typically specified using the index parameter in the constructor.

```
ser = pd.Series([5,0,3,8,4], index=['red','blue','yellow','white','green'])
```

```
ser.index
```

```
Index(['red', 'blue', 'yellow', 'white', 'green'], dtype='object')
```

• Methods on Index

- `idmin()` and `idmax()` are two functions that return, respectively, the index with the lowest value and the index with the highest value.

```
ser.index  
Index(['red', 'blue', 'yellow', 'white', 'green'], dtype='object')
```

```
S  
ser.idxmin()  
'blue'  
  
S  
ser.idxmax()  
'white'
```

• Index with Duplicate Labels

```
serd = pd.Series(range(6),  
index=['white', 'white', 'blue', 'green', 'green', 'yellow'])
```

serd

	0
white	0
white	1
blue	2
green	3
green	4
yellow	5

dtype: int64

Regarding the selection of elements in a data structure, if there are more values with the same label, you get a series in place of a single element.

```
serd['white']
```

	0
white	0
white	1

dtype: int64

When data structures grow larger, detecting duplicate index labels becomes difficult. To address this, pandas provides the `is_unique` attribute in Index objects, which checks whether a Series or DataFrame contains any duplicate index labels

```
serd.index.is_unique
```

False

```
frame.index.is_unique
```

True

Other Functionalities on Indexes

Reindexing

- Although an **Index object is immutable** once created, you can effectively change it through **reindexing**. Reindexing generates a new data structure based on the original one, allowing you to redefine the indexing rules.

```
ser = pd.Series([2,5,7,4], index=['one', 'two', 'three', 'four'])
```

ser

	0
one	2
two	5
three	7
four	4

dtype: int64

```
ser.reindex(['three', 'four', 'five', 'one'])
```

	0
three	7.0
four	4.0
five	NaN
one	2.0

dtype: float64

- Reindexing can rearrange label order, remove existing labels, and introduce new ones (which may result in missing values). Manually specifying all labels can be inconvenient for large datasets, so pandas provides methods to **automatically fill or interpolate values** during reindexing, making the process more efficient and practical.
- When an index has missing values (such as gaps in a numeric sequence), you can use **reindexing with the method='ffill' option** to fill in those gaps. By specifying a complete index range (e.g., range(6) for values 0 to 5), pandas forward-fills missing entries using the previous available value, helping create a continuous sequence.

```
ser3 = pd.Series([1,5,6,3],index=[0,3,5,6])
```

ser3

0	
0	1
3	5
5	6
6	3

dtype: int64

```
ser3.reindex(range(6),method='ffill')
```

0	
0	1
1	1
2	1
3	5
4	5
5	6

dtype: int64

- When an index has missing values (such as gaps in a numeric sequence), you can use **reindexing with the method='ffill' option** to fill in those gaps. By specifying a complete index range (e.g., range(6) for values 0 to 5), pandas forward-fills missing entries using the previous available value, helping create a continuous sequence.
- When reindexing with interpolation, missing indexes are added and filled based on nearby values. Using **forward fill (ffill)**, missing indexes take the previous available value (e.g., indexes 1 and 2 receive the value from index 0). If you want missing indexes to take the next available value instead, you should use **backward fill (bfill)**.

```
ser3.reindex(range(6),method='ffill')
```

```
0  
0 1  
1 1  
2 1  
3 5  
4 5  
5 6
```

dtype: int64

```
ser3.reindex(range(6),method='bfill')
```

```
0  
0 1  
1 5  
2 5  
3 5  
4 6  
5 6
```

dtype: int64

- Using **backward fill (bfill)**, missing indexes (such as 1 and 2) are filled with the next available value (from index 3, value 5). When extending reindexing from a Series to a **DataFrame**, you can rearrange or redefine **rows, columns, or both**. Any newly added rows or columns that lack original data are automatically filled with **NaN** values.

```
frame.reindex(range(5), method='ffill',  
              columns=['colors', 'price', 'new', 'object'])
```

	item	colors	price	new	object
	id				
0		blue	1.2	blue	ball
1		green	1.0	green	pen
2		yellow	2.2	yellow	pencil
3		red	0.9	red	paper
4		white	1.7	white	mug

• Dropping

- function `drop()` to delete it.

```
ser = pd.Series(np.arange(4.),  
                index=['red', 'blue', 'yellow', 'white'])
```

ser

```
0  
red    0.0  
blue   1.0  
yellow 2.0  
white  3.0  
  
dtype: float64
```

```
ser.drop('yellow')
```

```
0  
red    0.0  
blue   1.0  
white  3.0  
  
dtype: float64
```

```
ser.drop(['blue', 'white'])
```

```
0  
red    0.0  
yellow 2.0  
  
dtype: float64
```


- In a DataFrame, values can be removed by referencing the **labels of either axis** (rows or columns). This allows you to delete specific rows or columns based on their labels.

```
frame = pd.DataFrame(np.arange(16).reshape((4,4)),  
... index=['red','blue','yellow','white'],  
... columns=['ball','pen','pencil','paper'])
```

frame

	ball	pen	pencil	paper
red	0	1	2	3
blue	4	5	6	7
yellow	8	9	10	11
white	12	13	14	15

```
frame.drop(['blue','yellow'])
```

	ball	pen	pencil	paper
red	0	1	2	3
white	12	13	14	15

```
frame.drop(['pen','pencil'],axis=1)
```

	ball	paper
red	0	3
blue	4	7
yellow	8	11
white	12	15

Arithmetic and Data Alignment

- One of pandas' most powerful features is its ability to **automatically align indexes** from different data structures, especially during arithmetic operations. Even if indexes are in different orders or exist in only one structure, pandas matches data based on their labels. This makes operations between Series or DataFrames with partially matching indexes flexible and efficient.

```
s1 = pd.Series([3,2,5,1],['white','yellow','green','blue'])
```

```
s2 = pd.Series([1,4,7,2,1],['white','yellow','black','blue','brown'])
```

- When performing arithmetic operations like addition between two Series, pandas aligns data by their labels. If a label exists in both Series, their corresponding values are added. If a label appears in only one Series, it still appears in the result, but its value is set to **NaN**.

s1		s2		s1 + s2	
	0		0		0
white	3	white	1	black	NaN
yellow	2	yellow	4	blue	3.0
green	5	black	7	brown	NaN
blue	1	blue	2	green	NaN
		brown	1	white	4.0
dtype: int64		dtype: int64		yellow	6.0
				dtype: float64	

- In the case of the dataframe, although it may appear more complex, the alignment follows the same principle, but is carried out both for the rows and for the columns.

```
frame1 = pd.DataFrame(np.arange(16).reshape((4,4)),  
... index=['red','blue','yellow','white'],  
... columns=['ball','pen','pencil','paper'])
```

```
frame2 = pd.DataFrame(np.arange(12).reshape((4,3)),  
... index=['blue','green','white','yellow'],  
... columns=['mug','pen','ball'])
```

frame1

	ball	pen	pencil	paper
red	0	1	2	3
blue	4	5	6	7
yellow	8	9	10	11
white	12	13	14	15

frame2

	mug	pen	ball
blue	0	1	2
green	3	4	5
white	6	7	8
yellow	9	10	11

frame1 + frame2

	ball	mug	paper	pen	pencil
blue	6.0	NaN	NaN	6.0	NaN
green	NaN	NaN	NaN	NaN	NaN
red	NaN	NaN	NaN	NaN	NaN
white	20.0	NaN	NaN	20.0	NaN
yellow	19.0	NaN	NaN	19.0	NaN

Operations Between Data Structures

- **Flexible Arithmetic Methods:**
 - `add()`, `sub()`, `div()`, `mul()`

```
frame1.add(frame2)
```

	ball	mug	paper	pen	pencil
blue	6.0	NaN	NaN	6.0	NaN
green	NaN	NaN	NaN	NaN	NaN
red	NaN	NaN	NaN	NaN	NaN
white	20.0	NaN	NaN	20.0	NaN
yellow	19.0	NaN	NaN	19.0	NaN

• Operations Between Dataframes and Series

- Coming back to the arithmetic operators, pandas allows you to make transactions between different structures, such as between a dataframe and a series.

```
frame = pd.DataFrame(np.arange(16).reshape((4,4)),  
... index=['red','blue','yellow','white'],  
... columns=['ball','pen','pencil','paper'])
```

frame

	ball	pen	pencil	paper
red	0	1	2	3
blue	4	5	6	7
yellow	8	9	10	11
white	12	13	14	15

```
ser = pd.Series(np.arange(4),  
... index=['ball','pen','pencil','paper'])
```

ser

```
0  
ball 0  
pen 1  
pencil 2  
paper 3
```

dtype: int64

frame - ser

	ball	pen	pencil	paper
red	0	0	0	0
blue	4	4	4	4
yellow	8	8	8	8
white	12	12	12	12

Function Application and Mapping

Functions by Element

- pandas is built on top of NumPy and extends many of its features to work with Series and DataFrame structures. One important feature it inherits is **universal functions (ufuncs)**, which operate element by element across the data structure.

```
frame = pd.DataFrame(np.arange(16).reshape((4,4)),  
... index=['red','blue','yellow','white'],  
... columns=['ball','pen','pencil','paper'])
```

frame

	ball	pen	pencil	paper
red	0	1	2	3
blue	4	5	6	7
yellow	8	9	10	11
white	12	13	14	15

```
# calculate the square root of each value in the dataframe  
#using the NumPy np.sqrt()  
np.sqrt(frame)
```

	ball	pen	pencil	paper
red	0.000000	1.000000	1.414214	1.732051
blue	2.000000	2.236068	2.449490	2.645751
yellow	2.828427	3.000000	3.162278	3.316625
white	3.464102	3.605551	3.741657	3.872983

Functions by Row or Column

- In pandas, you can apply not only built-in universal functions (ufuncs) but also **user-defined functions**. These functions typically operate on a one-dimensional array and return a single value. For example, a lambda function can be created to calculate the range (difference between maximum and minimum values) of elements in an array.

frame				
	ball	pen	pencil	paper
red	0	1	2	3
blue	4	5	6	7
yellow	8	9	10	11
white	12	13	14	15

```
#define a lambda function that calculates  
#the range covered by the elements in an array  
f = lambda x: x.max() - x.min()
```

```
#define the function  
def f(x):  
    return x.max() - x.min()
```

```
#Using the apply() function  
#apply the function just defined on the dataframe  
frame.apply(f)
```

```
0  
ball 12  
pen 12  
pencil 12  
paper 12  
dtype: int64
```


- A useful case is to extend the application to many functions simultaneously. In this case, you have two or more values for each feature applied.

```
def f(x):  
    return pd.Series([x.min(), x.max()], index=['min', 'max'])
```

```
frame.apply(f)
```

	ball	pen	pencil	paper
min	0	1	2	3
max	12	13	14	15

Statistics Functions

- `sum()`, `mean()`, ...

```
frame.sum()
```

	0
ball	24
pen	28
pencil	32
paper	36

dtype: int64

```
frame.mean()
```

	0
ball	6.0
pen	7.0
pencil	8.0
paper	9.0

dtype: float64

```
frame.describe()
```

	ball	pen	pencil	paper
count	4.000000	4.000000	4.000000	4.000000
mean	6.000000	7.000000	8.000000	9.000000
std	5.163978	5.163978	5.163978	5.163978
min	0.000000	1.000000	2.000000	3.000000
25%	3.000000	4.000000	5.000000	6.000000
50%	6.000000	7.000000	8.000000	9.000000
75%	9.000000	10.000000	11.000000	12.000000
max	12.000000	13.000000	14.000000	15.000000

Sorting and Ranking

- Sorting is an important operation in data analysis, and pandas makes it easy using the **sort_index()** function. This function returns a new object with the same data as the original, but with the elements ordered according to their index.

```
ser = pd.Series([5,0,3,8,4],  
... index=['red','blue','yellow','white','green'])
```

ser

	0
red	5
blue	0
yellow	3
white	8
green	4

dtype: int64

```
ser.sort_index()
```

	0
blue	0
green	4
red	5
white	8
yellow	3

dtype: int64

```
ser.sort_index(ascending=False)
```

	0
yellow	3
white	8
red	5
green	4
blue	0

dtype: int64

- In a DataFrame, sorting can be applied separately to either axis. To sort by **rows (index)**, use `sort_index()` with default settings. To sort by **columns**, set the parameter `axis=1` in the `sort_index()` function.

```
frame = pd.DataFrame(np.arange(16).reshape((4,4)),  
                      index=['red', 'blue', 'yellow', 'white'],  
                      columns=['ball', 'pen', 'pencil', 'paper'])
```

frame

	ball	pen	pencil	paper
red	0	1	2	3
blue	4	5	6	7
yellow	8	9	10	11
white	12	13	14	15

```
frame.sort_index()
```

	ball	pen	pencil	paper
blue	4	5	6	7
red	0	1	2	3
white	12	13	14	15
yellow	8	9	10	11

```
frame.sort_index(axis=1)
```

	ball	paper	pen	pencil
red	0	3	1	2
blue	4	7	5	6
yellow	8	11	9	10
white	12	15	13	14

- The `sort_values()` function

```
frame.sort_values(by='pen')
```

	ball	pen	pencil	paper
red	0	1	2	3
blue	4	5	6	7
yellow	8	9	10	11
white	12	13	14	15

```
frame.sort_values(by=['pen', 'pencil'])
```

	ball	pen	pencil	paper
red	0	1	2	3
blue	4	5	6	7
yellow	8	9	10	11
white	12	13	14	15



- **Ranking** is closely related to sorting and involves assigning a sequential rank to each element in a Series. The ranking starts from the lowest value and increases progressively to the highest value

ser	
	0
red	5
blue	0
yellow	3
white	8
green	4
dtype: int64	

ser.rank()	
	0
red	4.0
blue	1.0
yellow	2.0
white	5.0
green	3.0
dtype: float64	

ser.rank(ascending=False)	
	0
red	2.0
blue	5.0
yellow	4.0
white	1.0
green	3.0
dtype: float64	

Correlation and Covariance

- Two important statistical calculations are correlation and covariance, expressed in pandas by the **corr()** and **cov()** functions. These kinds of calculations normally involve two series.

```
seq1 = pd.Series([1,2,3,4,4,3,2,1],  
                 ['2006','2007','2008','2009',  
                 '2010','2011','2012','2013'])
```

seq1

	0
2006	1
2007	2
2008	3
2009	4
2010	4
2011	3
2012	2
2013	1

dtype: int64

```
seq2 = pd.Series([3,4,3,4,5,4,3,2],  
                 ['2006','2007','2008','2009',  
                 '2010','2011','2012','2013'])
```

seq2

	0
2006	3
2007	4
2008	3
2009	4
2010	5
2011	4
2012	3
2013	2

dtype: int64

```
seq1.corr(seq2)
```

```
np.float64(0.7745966692414835)
```

```
seq1.cov(seq2)
```

```
np.float64(0.8571428571428571)
```

- Covariance and correlation can also be applied to a single dataframe. In this case, they return their corresponding matrices in the form of two new dataframe objects.

```
frame2 = pd.DataFrame([[1,4,3,6],[4,5,6,1],[3,3,1,5],[4,1,6,4]],  
index=['red','blue','yellow','white'],  
columns=['ball','pen','pencil','paper'])
```

frame2

	ball	pen	pencil	paper
red	1	4	3	6
blue	4	5	6	1
yellow	3	3	1	5
white	4	1	6	4

frame2.corr()

	ball	pen	pencil	paper
ball	1.000000	-0.276026	0.577350	-0.763763
pen	-0.276026	1.000000	-0.079682	-0.361403
pencil	0.577350	-0.079682	1.000000	-0.692935
paper	-0.763763	-0.361403	-0.692935	1.000000

frame2.cov()

	ball	pen	pencil	paper
ball	2.000000	-0.666667	2.000000	-2.333333
pen	-0.666667	2.916667	-0.333333	-1.333333
pencil	2.000000	-0.333333	6.000000	-3.666667
paper	-2.333333	-1.333333	-3.666667	4.666667

“Not a Number” Data

- Missing data, represented by **NaN (Not a Number)**, is common in data analysis. pandas is specifically designed to handle these situations effectively. It provides tools to manage and treat missing values, and many operations—such as calculating descriptive statistics—automatically ignore NaN values to prevent errors and ensure accurate results

Assigning a NaN Value

- use the `np. nan` value of the NumPy library

```
ser = pd.Series([0,1,2,np.nan,9],  
                index=['red','blue','yellow','white','green'])
```

ser

	0
red	0.0
blue	1.0
yellow	2.0
white	NaN
green	9.0

dtype: float64

```
ser['white'] = None
```

ser

	0
red	0.0
blue	1.0
yellow	2.0
white	NaN
green	9.0

dtype: float64

Filtering Out NaN Values

- Using the `dropna()` function or `notnull()`.

ser	
	0
red	0.0
blue	1.0
yellow	2.0
white	NaN
green	9.0

ser.dropna()	
	0
red	0.0
blue	1.0
yellow	2.0
green	9.0
dtype: float64	

ser[ser.notnull()]	
	0
red	0.0
blue	1.0
yellow	2.0
green	9.0
dtype: float64	

- If you're dealing with a dataframe, it gets a little more complex. If you use the `dropna()` function on this type of object, and there is only one NaN value on a column or row, it will eliminate it.

```
frame = pd.DataFrame([[6,np.nan,6],[np.nan,np.nan,np.nan],[2,np.nan,5]],  
                      index = ['blue','green','red'],  
                      columns = ['ball','mug','pen'])
```

frame

	ball	mug	pen
blue	6.0	NaN	6.0
green	NaN	NaN	NaN
red	2.0	NaN	5.0

```
frame.dropna()
```

```
ball mug pen
```

```
#To avoid having entire rows and columns disappear completely  
#specify the how option, assigning a value of all to it.  
frame.dropna(how='all')
```

	ball	mug	pen
blue	6.0	NaN	6.0
red	2.0	NaN	5.0

Filling in NaN Occurrences

- Instead of removing NaN values—which might also eliminate useful data—you can replace them using the **fillna()** function. This method allows you to substitute all NaN values with a specified number, helping preserve the overall data structure during analysis.

```
frame.fillna(0)
```

	ball	mug	pen
blue	6.0	0.0	6.0
green	0.0	0.0	0.0
red	2.0	0.0	5.0

```
#can replace NaN with different values depending on the column,  
#specifying one by one the indexes and the associated values.  
frame.fillna({'ball':1,'mug':0,'pen':99})
```

	ball	mug	pen
blue	6.0	0.0	6.0
green	1.0	0.0	99.0
red	2.0	0.0	5.0

Hierarchical Indexing and Leveling

- Hierarchical indexing in pandas allows multiple levels of indexes on a single axis. This feature enables you to manage multi-dimensional data while still working within a two-dimensional structure like a Series or DataFrame.

```
mser = pd.Series(np.random.rand(8),  
                 index=[['white','white','white','blue','blue','red','red','red'],  
                       ['up','down','right','up','down','up','down','left']])
```

mser

		0
white	up	0.816885
	down	0.706274
	right	0.518810
blue	up	0.512942
	down	0.911262
red	up	0.160837
	down	0.707839
	left	0.561207

dtype: float64

mser.index

```
MultiIndex([( 'white',  'up'),  
            ( 'white',  'down'),  
            ( 'white',  'right'),  
            ( 'blue',   'up'),  
            ( 'blue',   'down'),  
            (  'red',   'up'),  
            (  'red',   'down'),  
            (  'red',   'left')],  
           )
```

- select the values for a given value of the first index

mser

		0
white	up	0.816885
	down	0.706274
	right	0.518810
blue	up	0.512942
	down	0.911262
red	up	0.160837
	down	0.707839
	left	0.561207

dtype: float64

mser['white']

	0
up	0.816885
down	0.706274
right	0.518810

dtype: float64

#select values for a given value of the second index
mser[:, 'up']

	0
white	0.816885
blue	0.512942
red	0.160837

dtype: float64

- Hierarchical indexing is essential for reshaping data and performing group-based operations like pivot tables. Using the **unstack()** function, a Series with a hierarchical index can be transformed into a DataFrame, where one level of the index is converted into a new set of columns.

```
mser.unstack()
```

	down	left	right	up
blue	0.911262	NaN	NaN	0.512942
red	0.707839	0.561207	NaN	0.160837
white	0.706274	NaN	0.51881	0.816885

- If you want to perform the reverse operation, which is to convert a dataframe to a series, use the **stack()** function.

```
frame1 = pd.DataFrame([[1,4,3,6],[4,5,6,1],[3,3,1,5],[4,1,6,4]],
    index=['red','blue','yellow','white'],
    columns=['ball','pen','pencil','paper'])
```

frame1

	ball	pen	pencil	paper
red	1	4	3	6
blue	4	5	6	1
yellow	3	3	1	5
white	4	1	6	4

frame1.stack()

```
0
red    ball    1
       pen     4
       pencil  3
       paper   6
blue   ball    4
       pen     5
       pencil  6
       paper   1
yellow ball    3
       pen     3
       pencil  1
       paper   5
white  ball    4
       pen     1
       pencil  6
       paper   4
```

dtype: int64

- In a DataFrame, hierarchical indexing can be applied to both rows and columns. When creating the DataFrame, you must provide arrays of arrays for the **index** and/or **columns** parameters to define multiple levels of indexing.

```
mframe = pd.DataFrame(np.random.randn(16).reshape(4,4),
    index=[['white', 'white', 'red', 'red'], ['up', 'down', 'up', 'down']],
    columns=[['pen', 'pen', 'paper', 'paper'], [1, 2, 1, 2]])
```

mframe

		pen		paper	
		1	2	1	2
white	up	0.523535	0.096620	0.891156	-0.736776
	down	-0.857183	0.655291	0.896131	-1.227288
red	up	-0.681045	-1.368256	1.499060	0.258799
	down	-0.143087	-0.019213	-0.143093	-1.940656

Reordering and Sorting Levels

- To rearrange the order of the levels on an axis or sort for values at a specific level.
- The **swaplevel()** function accepts as arguments the names assigned to the two levels that you want to interchange and returns a new object with the two levels interchanged between them, while leaving the data unmodified.

```
mframe.columns.names = ['objects','id']
```

```
mframe.index.names = ['colors','status']
```

mframe

objects		pen		paper	
id		1	2	1	2
colors		status			
white	up	0.523535	0.096620	0.891156	-0.736776
	down	-0.857183	0.655291	0.896131	-1.227288
red	up	-0.681045	-1.368256	1.499060	0.258799
	down	-0.143087	-0.019213	-0.143093	-1.940656

```
mframe.swaplevel('colors','status')
```

objects		pen		paper	
id		1	2	1	2
status		colors			
up	white	0.523535	0.096620	0.891156	-0.736776
down	white	-0.857183	0.655291	0.896131	-1.227288
up	red	-0.681045	-1.368256	1.499060	0.258799
down	red	-0.143087	-0.019213	-0.143093	-1.940656

- the **sort_index()** function orders the data considering only those of a certain level by specifying it as parameter

objects		pen		paper	
id		1	2	1	2
status	colors				
up	white	0.523535	0.096620	0.891156	-0.736776
down	white	-0.857183	0.655291	0.896131	-1.227288
up	red	-0.681045	-1.368256	1.499060	0.258799
down	red	-0.143087	-0.019213	-0.143093	-1.940656

```
mframe.sort_index(level='colors')
```

objects		pen		paper	
id		1	2	1	2
colors	status				
red	down	-0.143087	-0.019213	-0.143093	-1.940656
	up	-0.681045	-1.368256	1.499060	0.258799
white	down	-0.857183	0.655291	0.896131	-1.227288
	up	0.523535	0.096620	0.891156	-0.736776

Summary Statistics with groupby Instead of with Level

- Many descriptive statistics and summary statistics performed on a dataframe or on a series have still a level option, with which you can determine at what level the descriptive and summary statistics should be determined.

```
mframe.groupby(level='colors').sum()
```

objects	pen		paper	
id	1	2	1	2
colors				
red	-0.824132	-1.387470	1.355967	-1.681857
white	-0.333648	0.751911	1.787287	-1.964065

```
mframe.T.groupby(level='id').sum()
```

colors	white		red	
status	up	down	up	down
id				
1	1.414691	0.038948	0.818016	-0.286180
2	-0.640156	-0.571998	-1.109457	-1.959869

